DevMetrics

Our Project Journey

From "let's just fetch some GitHub data" to a full authenticated, encrypted, metrics-generating backend.

A story, not a spec. Written so you can pick this project back up and actually remember what you built and why.

0. Before We Start — What DevMetrics Actually Is

Okay, so let's sit down and go through this properly, from the top, like I'm just explaining it to you over coffee instead of you re-reading old code and panicking about what half of it does.

DevMetrics, at its core, is a pretty simple idea wearing a fairly serious backend. The idea is: a user connects their GitHub account, picks a repository, and DevMetrics goes and pulls all the commits, pull requests, reviews, and issues for that repo, crunches them into actual developer analytics, and eventually shows it all on a dashboard. That's it. That's the whole pitch.

But "just fetch GitHub data and show some numbers" turns into a real engineering project the second you ask two follow-up questions: *whose* GitHub data, and *who's allowed to see it*. Answering those two questions is basically the entire story of everything we built — authentication, a database, encrypted tokens, repository ownership checks, all of it. None of it was built because it sounded cool. Every layer showed up because the layer before it created a problem that needed solving.

So let's walk through it in the order it actually happened.

1. Day One — Just Talk to GitHub

Before there was a single user, a single password, or a single database table, DevMetrics couldn't even do the one thing it exists to do: talk to GitHub and get data back. So that's where we started — no auth, no database, nothing fancy. Just: can we hit the GitHub API and get something useful back?

That became the **GitHub service layer** — a file (`githubService.js`) full of small, focused functions, each one responsible for fetching exactly one kind of GitHub data:

```
fetchGithubRepo        → basic repo info
fetchForks             → who forked it
fetchContributors      → who contributes
fetchCommits           → commit history
fetchPullRequests      → PR history
fetchIssues            → issue history
fetchLanguages         → language breakdown
fetchCollaborators     → who has access
fetchBranches          → branch list
fetchIndividualCommit  → one commit in detail (stats!)
fetchIndividualPR      → one PR in detail
fetchPRReviews         → reviews on a PR
fetchPRComments        → comments on a PR
fetchIndividualIssue   → one issue in detail
fetchIssueTimeline     → the timeline of events on an issue
```

We tested every one of these individually before moving on. That might sound tedious, but it mattered — later on, when the metrics engine, authentication, and repository ownership were all wired together, if something broke we needed to already be 100% sure it wasn't one of these base functions. Getting the foundation verified early meant that every bug we hit later was a bug in something *new*, not something old creeping back.

> **The one architectural rule we set from day one:** the route should never contain GitHub API logic. The route's only job is to receive the HTTP request and hand it off. The service layer is the only thing that's allowed to know how to talk to GitHub. This separation felt like overkill on day one when there was no route yet — but it's exactly what let us bolt authentication, caching, authorization, and error handling on top later without rewriting the GitHub-fetching code itself.

2. Wiring It to Express — Routes

Once the service functions worked, we needed a way for the outside world (eventually, the frontend) to actually call them over HTTP. That's what the GitHub routes are for — they're the HTTP-facing skin on top of the service layer.

The flow became, and still is, dead simple:

```
HTTP Request
     │
     ▼
Express Server
     │
     ▼
GitHub Route      (githubRoute.js)
     │
     ▼
GitHub Service    (githubService.js)
     │
     ▼
GitHub API
```

And connecting it to the actual Express app was one line:

```
app.use('/api/github', githubRoute);
```

Now any request to `/api/github/...` gets forwarded straight to the GitHub router. This meant the frontend (whenever it exists) never has to talk to GitHub directly — it always goes through our backend. Which matters a lot once you realize the frontend is never going to be trusted with things like GitHub tokens, rate limit management, or caching logic. All of that stays server-side, hidden behind this one route prefix.

3. The GitHub Token — Why We Needed Real Credentials

Here's the thing about hitting GitHub's API without a token: it works, but you get slammed with rate limits almost immediately.

Unauthenticated requests get you a tiny quota, nowhere near enough for an app that's going to repeatedly pull commits, PRs, issues, and reviews for a repository. So we generated a GitHub Personal Access Token.

The obvious wrong way to do this is to just paste the token directly into `githubService.js`. We didn't do that. Instead, the token lives in the environment (`.env`), and `env.js` is the one file responsible for reading it out of `process.env` and exporting it cleanly to whatever needs it.

Then `githubService.js` was updated so every outgoing GitHub request carries it:

```
Authorization: Bearer <GitHub Token>
```

This is a small thing but it set the pattern for basically every credential we handled afterward: **credentials live in environment configuration, never in source code.** That rule comes back later, almost word for word, when we deal with the GitHub token encryption key.

4. The Cache — Because GitHub Doesn't Forget Rate Limits

Next we added a cache — nothing fancy, just an in-memory `Map` living in `cache.js`. The logic is exactly what you'd guess:

```
Need GitHub data
      │
      ▼
Is it already cached (and not expired)?
   ┌──────┴──────┐
  YES            NO
   │              │
Use cached     Call GitHub API
value              │
                Store in cache
                   │
                Return the data
```

The cache supports setting a value with an expiry, and reading it back. At this stage it mostly saved us from hammering GitHub during development and testing. But this decision turned out to matter a lot more later — once the metrics engine started firing off dozens of requests per repository (commits, individual commit stats, PRs, PR reviews...), avoiding repeated identical calls stopped being a nice-to-have and became the difference between "this works" and "this trips GitHub's rate limiter constantly."

With the service layer, the routes, the token, and the cache in place, the backend could officially do its first real job:

```
server.js
   │
   ▼
githubRoute.js
   │
   ▼
githubService.js
   │
   ▼
GitHub API
```

That's the whole backbone. Everything from here on is either *protecting* that backbone (auth, encryption, ownership checks) or *building on top of* it (the metrics engine).

5. Why We Needed a Database at All

Here's the question that ended the "just fetch GitHub data" phase of the project: *whose* repository is this? GitHub itself doesn't know or care about DevMetrics users. If we wanted multiple people using this app, each connecting their own GitHub account and picking their own repos, we needed somewhere to actually remember that — who's who, which GitHub account belongs to which DevMetrics user, and which repositories they've chosen to track.

That's the whole reason PostgreSQL and Prisma entered the picture. Not because a database is "standard," but because DevMetrics genuinely couldn't be a multi-user app without persisting that ownership information somewhere.

We landed on three models, and the relationship between them tells you almost the whole story of the app's data model:

```
User
  │
  │ one-to-one
  ▼
GithubAccount
  │
  │ one-to-many
  ▼
Repository
```

One `User` has (at most) one connected `GithubAccount`. That GithubAccount can have many `Repository` rows — one for each repo the user has chosen to track. In Prisma, roughly:

```
model User {
  id            Int      @id @default(autoincrement())
  name          String
  email         String   @unique
  passwordHash  String
  githubAccount GithubAccount?
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}

model GithubAccount {
  id             Int          @id @default(autoincrement())
  githubUsername String
  encyptedToken  String
  userId         Int          @unique
  user           User         @relation(fields: [userId], references: [id])
  repositories   Repository[]
}

model Repository {
  id             Int           @id @default(autoincrement())
  githubRepoId   Int
  repoName       String
  fullName       String
  githubAccountId Int
  githubAccount  GithubAccount @relation(fields: [githubAccountId], references: [id])

  @@unique([githubAccountId, githubRepoId])
}
```

That last line — the `@@unique` on `[githubAccountId, githubRepoId]` — was a deliberate decision, not an afterthought. It means the database itself refuses to let the same GitHub repository be added twice under the same account. We didn't want to rely on remembering to check for duplicates in application code every single time; we wanted the database to make it structurally impossible.

6. The JavaScript/TypeScript Friction

Here's a wrinkle that isn't really about databases at all, it's about the fact that the backend was written in plain JavaScript, while Prisma's generated layer leans on TypeScript. So there was a real question of: can our JavaScript backend actually run alongside this TypeScript-based database layer without everything falling apart?

The answer was yes, but it needed a bit of care — specifically, the server had to actually be run through a tool that understands TypeScript on the fly:

```
npx tsx server.js
```

instead of a plain `node server.js`. It's a small detail, but it's exactly the kind of thing that, if you forget it, gives you a confusing wall of errors that look like they're about your code when they're really about how the process was started.

7. DATABASE_URL and Actually Standing Up Postgres

Before Prisma can do anything, it needs to know where the actual database lives. That's the `DATABASE_URL` environment variable, something like:

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/new_db"
```

We got PostgreSQL running locally, connected pgAdmin to it so we could actually see inside the database while developing, and pointed `DATABASE_URL` at it. Nothing glamorous here — this step is just plumbing — but it's the plumbing everything else depends on.

8. Prisma Contracts

Once the schema existed, we needed Prisma to actually generate the client code our backend would import and call. That's done through:

```
npm run contract:emit
```

which produces `contract.json` and `contract.d.ts`. Think of this step as Prisma reading our schema and generating a typed, ready-to-use client so the rest of the backend isn't writing raw SQL by hand — it's calling normal-looking JavaScript functions that Prisma turns into safe SQL under the hood.

9. The Migration Problem

This is where things got genuinely annoying for a bit, and it's worth walking through because the lesson is a good one.

> **WHAT BROKE**
> Running `npm run migration:plan` threw errors saying migration-related tables already existed. Not the models we designed — Prisma's own internal bookkeeping tables that track which migrations have already been applied.

The instinct here could easily have been "just delete everything and start fresh." We didn't do that. Instead we actually investigated what state the migration history was in, and fixed the mismatch by explicitly pointing Prisma's migration tracking at the correct starting point:

```
npx prisma migration ref set db 1e8412e162dbbe69f4bb3bf8d07f0280ae67eaab15c34dcf201e67468315428d
```

> **WHAT FIXED IT**
> After resetting that reference point, `npm run migration:plan` ran cleanly. We also used `npx prisma db update --dry-run` along the way, specifically so we could see what Prisma *wanted* to change before actually letting it touch the real database.

The real lesson from this stage: **the database's actual schema and Prisma's record of what migrations have run are two separate things**, and they can get out of sync. When that happens, the fix isn't to nuke the database — it's to reconcile the migration history with reality.

10. User Registration

With the database finally solid, we could build the first real feature: registration. We created an `authRoute`, and inside it, `/auth/register`.

The flow, in order:

```
Receive name / email / password
        |
        ▼
Are all three fields present?
        |
        ▼
Does this email already exist?
        |
        ▼
Hash the password (Argon2)
        |
        ▼
```

```
    Create the user row
```

Each of those checks exists because of a specific failure mode: missing fields would crash downstream code expecting them, and duplicate emails would silently create two accounts sharing an identity in the real world but not in our database. Simple checks, but they're the difference between an API that fails predictably and one that fails weirdly three steps later.

11. Argon2 — Why We Hash Passwords Instead of Encrypting Them

This is a distinction that matters a lot, and it comes up again almost immediately when we get to GitHub tokens, so it's worth being precise about it now.

For a password, we never, ever need to get the original value back. Nobody at DevMetrics should ever need to know what a user's actual password is — all we need to know is "does this login attempt match what they originally set." That's a one-way problem, and Argon2 is built exactly for that:

```
password
   │
   ▼
Argon2
   │
   ▼
hash  (stored — the original password is gone forever)
```

Because it's one-way, even if our entire database leaked, an attacker doesn't get the passwords back — they get hashes that are (with a strong algorithm like Argon2) extremely expensive to reverse. That's exactly the property we want for something we never need to un-hide.

12. Login and Setting Up JWTs

Login is basically registration's mirror image: take the email and password, look the user up, and check the supplied password against the stored Argon2 hash.

```
Login request
     │
     ▼
Find user by email
     │
     ▼
Verify password against Argon2 hash
     │
     ▼
Issue a JWT
```

Once the password checks out, we need to give the user something that proves, on every future request, "yes, this is a logged-in user, and here's who they are" — without making them log in again on every single request. That's what the JWT is for.

We generated the signing secret with Node's own crypto module:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

and chose **HS256** as the signing algorithm specifically because it's symmetric — the exact same secret both signs new tokens and verifies incoming ones. There's no public/private key pair to manage here; one secret does both jobs, which is exactly the right amount of complexity for a single backend service that issues and checks its own tokens.

The token's lifetime went through one adjustment worth mentioning: it started at one hour, which turned out to be annoyingly short for actually using the app during development, and the plan is to move it to something closer to five days. Small detail, but it's a reminder that "how long should a session live" is a real design decision, not a default you leave untouched.

13. Authentication Middleware

Now that logging in produces a JWT, we needed a gatekeeper — something that checks that token on every protected route, instead of copy-pasting the same verification logic into every single route handler.

```
Incoming request
      │
      ▼
Read Authorization header
      │
      ▼
Is a token present?
      │
      ▼
Verify it against the JWT secret
      │
      ▼
```

```
Extract userId from the payload
      |
      ▼
Let the request through
```

If any of those steps fail — no token, bad signature, expired token — the request gets stopped right there and never reaches the actual route logic. This is the piece that made it possible to say things like "only a logged-in user can connect a GitHub account" or "only the owner of a repository can fetch its metrics," because every protected route can now just assume `req.userId` already exists and is trustworthy by the time it runs.

We tested this middleware directly — hitting a protected route with no token, with a bad token, and with a valid one — before building anything on top of it. Once it held up under all three, we moved forward with confidence that the gatekeeper actually worked.

14. Connecting a GitHub Account

Now for the question this whole authentication layer existed to answer: once someone is logged into DevMetrics, how do we know which GitHub account is *theirs*?

We built `/connect`. The user supplies their GitHub username and a GitHub token. The backend doesn't just trust that blindly — it goes and asks GitHub, using that token, who it actually belongs to, and checks that the answer matches the username the user typed in.

```
Logged-in DevMetrics user
          |
          ▼
      POST /connect  (username + token)
          |
          ▼
   Ask GitHub: who does this token belong to?
          |
          ▼
Does GitHub's answer match the submitted username?
     ┌────┴────┐
    YES        NO
     |          |
  Store it   Reject with an error
```

If the token is invalid, or the username the user typed doesn't match what GitHub actually says the token belongs to, the connection is rejected. This step is what turned DevMetrics from "a thing that can call the GitHub API" into "a real multi-user application" — because from this point on, a GitHub identity is permanently tied to a specific DevMetrics account.

We also added a check to stop someone from connecting a second GitHub account when they already had one. One user, one GitHub identity, at least for now — that's a deliberate simplification, not a limitation we hit by accident.

> **WHAT BROKE, ALONG THE WAY**
> During this stage we hit a batch of annoying-but-real bugs: rows we expected to be inserted into Postgres just weren't showing up, even though the connect/login/protected routes all appeared to be working. The duplicate-account check also wasn't firing correctly the second time `/connect` was hit. And separately, `npm run dev` wasn't reliably keeping the server alive during development. These got worked through one at a time rather than papered over — the point being, at this stage in the project, "it looks like it works" and "it actually persists correctly" are two different claims, and it's worth verifying both.

15. The Big Security Decision — Hash the Password, Encrypt the Token

Once GitHub tokens started living in our database, we hit the second half of that hashing-vs-encryption distinction from earlier, and it's worth stating plainly because it's the kind of thing that sounds like a minor detail but is actually a core security decision:

Password	GitHub Token
Never needs to be recovered	Must be recoverable — DevMetrics needs the real token to call the GitHub API on the user's behalf
One-way: `password → Argon2 → hash`	Two-way: `token → encrypt → encrypted token`, and later `encrypted token → decrypt → token`

That's the whole reasoning. Hashing throws information away on purpose. Encryption keeps the information intact, just locked, because we know for a fact we'll need to unlock it again later.

16. Encrypting the GitHub Token — AES-256-GCM, Explained Like a Friend

So here's what we were actually trying to protect. Without encryption, PostgreSQL would be sitting there holding something like `github_pat_11ABC123XYZ...` in plain text. Anyone who got read access to that database — a leaked backup, a misconfigured permission, whatever — would have a live, usable GitHub credential in their hands. We wanted the database to instead hold something useless on its own, like `a91f2c...:7d82ab...:e91a4c...` — meaningless without a separate secret key that isn't stored anywhere near it.

We put the actual cryptography in its own file, `backend/utils/encryption.js`, specifically so that `githubRoute.js` never has to know or care how AES works. The route just calls `encrypt(githubToken)` and gets a string back.

```
import crypto from "crypto";
import { GITHUB_TOKEN_ENCRYPTION_KEY } from "../config/env.js";

const ALGORITHM = "aes-256-gcm";
const KEY = Buffer.from(GITHUB_TOKEN_ENCRYPTION_KEY, "hex");

if (KEY.length !== 32) {
  throw new Error("GITHUB_TOKEN_ENCRYPTION_KEY must be exactly 32 bytes");
}

export function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv(ALGORITHM, KEY, iv);
  let encrypted = cipher.update(text, "utf8", "hex");
  encrypted += cipher.final("hex");
  const authTag = cipher.getAuthTag();
  return [iv.toString("hex"), authTag.toString("hex"), encrypted].join(":");
}
```

Let's actually go through what each piece is doing, because none of this is magic.

AES-256 — the algorithm and the key size

AES is just a well-established, widely trusted symmetric encryption algorithm — "symmetric" meaning the same secret key both locks and unlocks the data. The "256" is the key size in bits. 256 bits works out to exactly 32 bytes, which is exactly why the code throws an error the moment `KEY.length !== 32`. That check isn't paranoia, it's **fail-fast configuration validation** — if the key is the wrong size, we want the app to refuse to start rather than silently run with broken encryption.

Turning a hex string into an actual key

The key itself was generated the same way as the JWT secret:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

`randomBytes(32)` gives 32 truly random bytes, and `.toString("hex")` turns that into a 64-character hex string — a format that's safe to paste into a `.env` file. Later, `Buffer.from(..., "hex")` converts that same string back into the actual 32-byte buffer Node's crypto functions expect. So the round trip looks like:

```
32 random bytes → hex encoding → 64-char .env value → Buffer.from(hex) → 32-byte key again
```

The IV — Initialization Vector

This one trips people up the first time. The IV is a random value generated fresh for every single encryption operation — here, 12 random bytes. It's not secret, and it doesn't need to be; it gets stored right alongside the encrypted output. Its job is different: without it, encrypting the exact same token twice would produce the exact same ciphertext both times, which leaks information (an attacker could tell "these two rows have the same token" just by looking). With a random IV every time, the same token encrypted twice produces two completely different-looking outputs. The key is the secret. The IV just guarantees uniqueness.

The authentication tag

This is the reason we specifically picked GCM mode over plain AES. When encryption finishes, GCM hands back an authentication tag — essentially a fingerprint of the ciphertext. When we later decrypt, if even a single byte of the stored data has been tampered with, that tag won't match and decryption fails loudly instead of quietly returning corrupted garbage. So GCM buys us two things at once: the data is unreadable without the key (confidentiality), and we can detect if it's been messed with (integrity).

Why we store three things together

The final encrypted value is `IV : AuthTag : EncryptedData`, all joined with colons into one string. We store the IV because we need it again for decryption. We store the auth tag because we need it to verify nothing was tampered with. And obviously we store the encrypted data itself. All three of these are safe to sit next to each other in the database, because none of them is the actual secret — the secret is the encryption key sitting in `.env`, completely separate from PostgreSQL. If someone steals the database, they get three useless strings. If someone steals `.env` too, then we have a much bigger problem — which is exactly why `.env` stays out of git entirely.



17. The Encryption Key Bug — A Real Debugging Story

WHAT BROKE

After finishing the encryption work and restarting the server with `npm run dev`, this showed up immediately:

```
Error: GITHUB_TOKEN_ENCRYPTION_KEY must be exactly 32 bytes
    at .../backend/utils/encryption.js:12:11
```

The cryptography code hadn't changed. It just started throwing this error on a plain restart, which is exactly the kind of thing that makes you doubt code you were sure worked five minutes ago.

The instinct might be to start questioning AES, the hex conversion, the buffer length math — all the crypto stuff we just walked through. But none of that was actually broken. The bug was sitting one layer up, in `env.js`. It was reading:

```
process.env.GITHUB_TOKEN
```

instead of:

```
process.env.GITHUB_TOKEN_ENCRYPTION_KEY
```

WHAT FIXED IT

Once `env.js` was pointed at the correct variable name, `encryption.js` finally received the real 64-character hex string we'd generated, and the 32-byte length check passed as expected.

The actual lesson here is bigger than this one typo: **when a cryptographic component complains that a key is the wrong size, don't start debugging the cryptography — trace the entire configuration path first.** The math around AES keys is fixed and well understood; it almost never lies to you. Environment variable plumbing, on the other hand, is exactly the kind of thing that quietly breaks when you're juggling multiple similarly-named `.env` keys (`GITHUB_TOKEN` for the API token, `GITHUB_TOKEN_ENCRYPTION_KEY` for the encryption key — easy to see how those get crossed).

18. Decryption

With encryption solid, decryption was the natural next addition to the same file — because encrypting a token is only half the story. Eventually, when DevMetrics actually needs to call the GitHub API on a user's behalf, it has to go the other direction:



This is exactly why the encryption work had to happen before repository selection could actually be wired up to real GitHub calls — there's no point letting a user pick a repository to analyze if we have no way to get a usable token back out of storage to actually fetch that repository's data.

One honest note for later: the plan going forward is to revisit and harden all of this — hashing, encryption, decryption, the works — once the full backend and frontend are actually working together end to end. Getting the mechanism correct first, then hardening it, was the deliberate order here.

19. Repository Fetching and Selection

With a connected, decryptable GitHub account in place, the next problem was: which repository does the user actually want DevMetrics to analyze? A user might have dozens of repos — we needed a way to list them, and a way to pick one.

Three endpoints came out of this stage:

/repositories

Fetches every repository belonging to the connected GitHub account, with checks in place to make sure the request is actually coming from the account owner. This one worked cleanly and became the "browse your repos" endpoint.

/select

Lets the user pick one of those repositories to track.

> **WHAT BROKE**
> A naming mismatch — one part of the code referred to `repoName`, another to `reponame` — caused the selection to fail. Casing mismatches like this are sneaky because JavaScript won't warn you; you just get `undefined` silently flowing somewhere it shouldn't.

> **WHAT FIXED IT**
> This one was traced and fixed independently, by actually following the data through the request rather than guessing. It's a small bug, but it's worth calling out because it's the kind of debugging muscle that matters far more than any single framework — reading the actual field names being sent and compared, instead of assuming they matched.

/repositories/selected

Returns whichever repositories the user has already selected. Once this existed, the repository routes as a group were cleaned up and tightened into their more optimal current form.

20. Wiring the Selected Repository Into the GitHub Service Layer

This is the point where every previous piece finally clicks together into one working chain. Before this, we had authentication, a connected GitHub account, an encrypted token, and repository selection — but they were still somewhat separate concerns. Now they had to actually cooperate to answer one real request: "give me GitHub data for this user's selected repository."

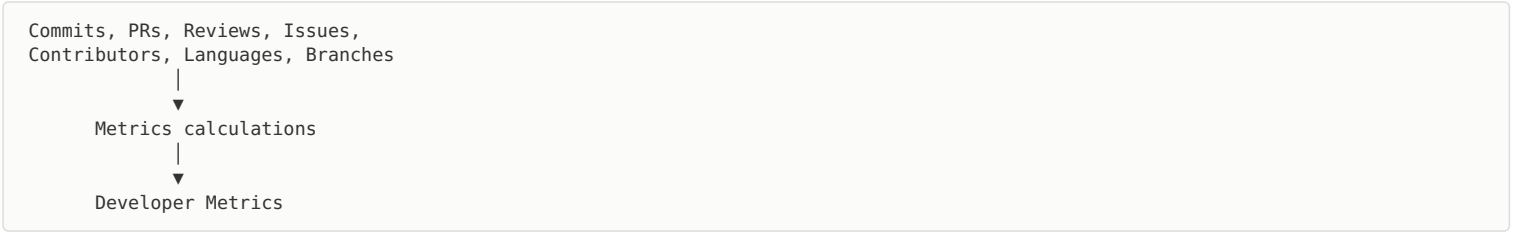
```
Authenticated user (via JWT)
        │
        ▼
Their connected GitHub account
        │
        ▼
Encrypted GitHub token → decrypt()
        │
        ▼
Selected repository
        │
        ▼
Authorization check:
does this repo actually belong to this user's account?
        │
        ▼
Existing GitHub service layer (from Section 1)
        │
        ▼
GitHub API
```

Notice the authorization check sitting right before the service layer gets called. That's deliberate. The question the backend has to ask is no longer just "give me this repository's data" — it's "is this *specific authenticated user* allowed to see *this specific repository's* data." Only after that check passes does the request get to reuse all the fetch functions we built and tested back on day one. This is the payoff of keeping the GitHub service layer dumb and reusable from the very beginning — it didn't need to

change at all to support multi-user authorization; everything new got layered around it.

21. The Metrics Engine — Turning Raw Data Into Something Meaningful

Everything up to this point has been about correctly and securely getting raw GitHub data for the right user and the right repository. But raw commits, PRs, issues, and reviews aren't DevMetrics yet — they're just GitHub's data, reshaped through our pipes. The actual product is what happens next: turning that raw data into developer metrics.



We built this in stages, each one its own focused module, rather than one giant function trying to do everything at once.

21.1 — Utility and Validation Functions

Before writing a single calculation, we built a layer of utility functions responsible for the boring-but-critical checks every calculation depends on. GitHub's data isn't guaranteed to be clean — fields can be missing, arrays can be empty, timestamps can be absent, numbers can come back as zero when you expected something else. This utility layer exists so every metric calculation downstream has a safer foundation to build on, instead of every single calculation re-implementing its own defensive checks. It's also the direct ancestor of the metric-validation hardening work described later — this is where "don't let bad data produce garbage numbers" starts.

21.2 — Commit Metrics

Takes the raw commit data for a repository and turns it into commit-related measurements — the kind of thing that later feeds both repository-level and developer-level metrics.

21.3 — Pull Request Metrics

Same idea, applied to pull requests — transforming raw PR data into measurements around how development activity actually flows through the repo.

21.4 — Review Metrics

Reviews are related to PRs but represent a distinct kind of activity — someone reviewing someone else's work rather than submitting their own — so they get their own dedicated calculation layer instead of being folded into PR metrics. This is also where a real performance problem started to surface: getting reviews for a PR requires a separate API call *per PR*. On a repository with a lot of pull requests, that's a lot of individual requests, and it's exactly the "PR review N+1 problem" that shows up again in the hardening section.

21.5 — Issue Metrics

The same raw-data-to-measurement pattern, applied to issues.

21.6 — Repository Metrics

This layer steps back and looks at the repository as a whole, rather than any single category of activity, combining what's needed to describe the repo's overall development activity.

21.7 — Developer-Level Aggregation

This is where the separate metric categories stop being separate. DevMetrics isn't really trying to reproduce GitHub's own API with extra steps — it's trying to answer questions about developers. So commit metrics, PR metrics, review metrics, issue metrics, and repository metrics all feed into an aggregation layer that produces per-developer results:



21.8 — The Metrics Orchestrator

Once all these individual pieces existed, something needed to coordinate the whole process — fetch the right GitHub data, run the right calculations in the right order, and combine the results into one final response. That's the orchestrator, living as the `index.js` of the metrics module. The point of having a dedicated orchestrator is that none of the individual metric modules need to know about the rest of the application — they just do their one job when called. The orchestrator is the only thing that knows how all the pieces fit together.

21.9 — The Metrics API Route

With the orchestrator working, it got exposed over HTTP as:

```
/api/metrics/:owner/:repo
```

which pulls together everything built so far into one request path:

```
Request → Metrics route → Auth check → Repo + decrypted token
  → GitHub services → Metrics orchestrator → Metric modules
  → Final metrics response
```

21.10 — Testing It

Once this endpoint existed, we tested it — but not just "does it return something." The real question was whether the entire chain, from an HTTP request down through auth, decryption, GitHub calls, and every metric module, actually produced correct output end to end. That question is what led directly into the hardening phase.

22. Why Hardening Became Necessary

A metrics pipeline that works against a small, tidy test repository can look completely finished. Then you point it at a real repository with hundreds of commits and dozens of PRs, and several problems that were invisible before suddenly aren't.

Pagination

GitHub's API doesn't hand you an entire commit history in one response — it pages results. If DevMetrics only ever reads page one, a repository with 300 commits gets analyzed as if it only had the first 30 or 100. The metrics wouldn't be wrong exactly — they'd just be quietly, confidently incomplete, which is arguably worse than an error. The fix is straightforward in concept: keep requesting subsequent pages for commits, PRs, issues, contributors, and anything else paginated, until there's nothing left. The rule we're building toward is simple: **never silently analyze only page one.**

Commit statistics efficiency

The basic commit list from GitHub doesn't include line-level additions and deletions — getting those often requires a separate request per commit. That's the same shape of problem as pagination but multiplied per-commit, so it needs a deliberate strategy rather than a naive loop that fires one request per commit without regard for how many commits there are.

The PR review N+1 problem

Exactly the issue flagged back in Section 21.4: fetching reviews currently costs one API request per pull request. A hundred PRs means a hundred extra requests just for reviews. That doesn't scale, and it's one of the concrete things this hardening phase needs to fix.

GitHub rate limits

Once you're making pagination requests, per-commit stat requests, and per-PR review requests, you can rack up a real number of API calls fast. The backend needs to actually recognize when GitHub responds with a rate-limit error, rather than that error surfacing as a confusing, generic failure somewhere downstream.

Error handling

GitHub can hand back an invalid repository, an invalid token, a permissions error, or just an empty response. None of those should be allowed to crash the server. They need to become clean, predictable API error responses instead.

Metric validation

This connects directly back to the utility layer from Section 21.1. Missing or malformed GitHub data shouldn't be allowed to produce `NaN`, impossible negative numbers, or other nonsense in the final metrics. Calculations need to behave predictably even when the input isn't perfect.

A stable API response structure

Before a frontend gets built around `/api/metrics/:owner/:repo`, the shape of that response needs to stop shifting. Field names need to be locked in, and it needs to be clear which fields are always guaranteed to be present. Otherwise every backend tweak becomes a frontend-breaking change.

End-to-end testing

Finally, the hardening phase means testing the full pipeline under realistic conditions, not just individual functions in isolation:

Scenario	Expected result
No authentication	401
Invalid GitHub token	Controlled error, not a crash
Invalid repository	Controlled error
Empty repository	A valid (if mostly empty) metrics response
Repository with commits	Correct commit metrics
Repository with PRs	Correct PR / review metrics
Repository with issues	Correct issue metrics
Fully populated repository	Complete, correct response

This is the current phase of the project — proving that the whole pipeline holds up, not just that each individual piece works on its own.

23. What's Left — Frontend and the Dashboard

Once the backend is sufficiently hardened and the metrics API contract is stable, the remaining work shifts to the frontend:

```
Login UI
   │
   ▼
GitHub connection UI
   │
   ▼
Repository selection UI
   │
   ▼
Dashboard
   │
   ▼
Visualize DevMetrics
```

The goal for the end product hasn't changed since day one: DevMetrics is meant to be a real developer-analytics dashboard, not just a nicer way to look at raw GitHub API responses.

Where We Are Right Now

If you've lost the thread of this project again at some point in the future, this is the page to come back to.

The mental model, in one line

```
WHO ARE YOU?              → Authentication / JWT
WHICH GITHUB ACCOUNT?     → GitHub Account Connection
WHICH REPOSITORY?         → Repository Selection + Authorization
HOW DO WE ACCESS IT?      → Decrypt GitHub Token
GET THE DATA              → GitHub Services
WHAT DOES IT MEAN?        → Metrics Engine
HOW DO WE SHOW IT?        → Frontend Dashboard
```

What's completed

- ✔ GitHub service layer (all fetch functions, tested)
- ✔ GitHub routes, connected to Express
- ✔ GitHub Personal Access Token via env.js
- ✔ In-memory cache system
- ✔ PostgreSQL + Prisma, schema and migrations resolved
- ✔ User registration with Argon2 password hashing
- ✔ Login with JWT (HS256) issuance
- ✔ Authentication middleware, tested
- ✔ GitHub account connection (/connect), with duplicate-account protection
- ✔ GitHub token encryption and decryption (AES-256-GCM)
- ✔ Repository fetching, selection, and authorization checks
- ✔ Full metrics engine — commit, PR, review, issue, repository, and developer-level metrics
- ✔ Metrics orchestrator and the /api/metrics/:owner/:repo route

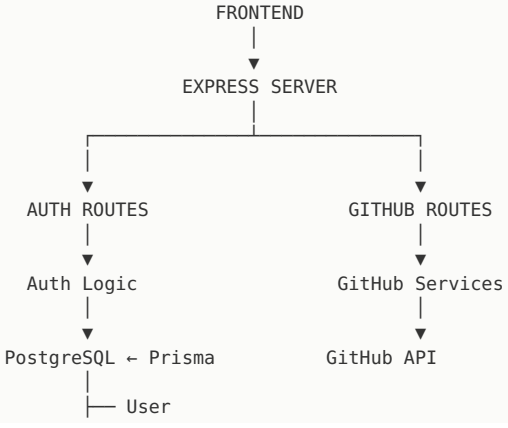
What's currently being hardened

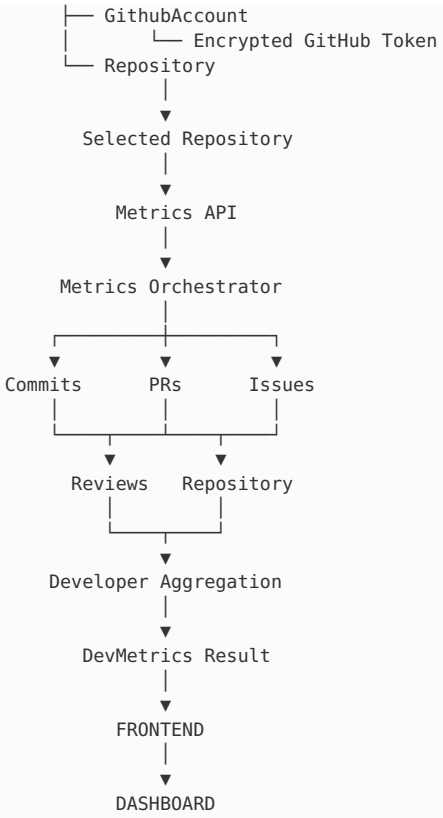
- ⚙ Pagination across commits, PRs, issues, and contributors
- ⚙ GitHub rate-limit detection and handling
- ⚙ The PR review N+1 request problem
- ⚙ Efficient commit statistics fetching
- ⚙ Clean, consistent error handling for GitHub failures
- ⚙ Metric validation (no NaN, no impossible values)
- ⚙ Locking down a stable metrics API response structure
- ⚙ Full end-to-end pipeline testing

What remains

- ○ Frontend: login, GitHub connection, and repository selection UI
- ○ The actual DevMetrics dashboard, visualizing everything the metrics engine produces

The current backend architecture





The next logical sequence

1. Finish the four-part hardening pass on the metrics system (fetching efficiency → reliability/error handling → stable response contract → end-to-end tests).
2. Lock the `/api/metrics/:owner/:repo` response shape so the frontend has something stable to build against.
3. Build the frontend flow in order: login → GitHub connection → repository selection → dashboard.
4. Revisit and harden the security layer (hashing, encryption, decryption) once frontend and backend are working together, as originally planned.

The project didn't get complicated by accident. Every layer showed up because the layer before it created a question that needed answering — GitHub services led to routes, routes led to needing real credentials, multiple users led to a database, a database led to needing to know who owns what, owning things led to needing to protect them, and protecting them is what finally made it safe to build the metrics engine on top. That's the whole story, in order.